

Phase-by-Phase Build Plan

Comprehensive Technical Implementation Guide

Version 1.0 | July 2026

Based on Technical Documentation v4.0

5 Phases | 10 Weeks | Next.js 15 + TypeScript + MySQL

Kenya Data Protection Act Compliant

Table of Contents

1	Executive Summary	3
2	Architecture Overview	4
3	Phase 1: Foundation and Infrastructure	6
4	Phase 2: Data Pipeline (CV and JD Processing)	9
5	Phase 3: Matching Engine	12
6	Phase 4: User Interface (Candidate Experience)	15
7	Phase 5: Admin Portal, Privacy, and Production	18
8	Cross-Phase Dependencies	20
9	Risk Register and Mitigations	21
10	Team Composition and Effort Estimates	22

1. Executive Summary

This document presents a structured, phase-by-phase build plan for the Job Matching Platform, a Kenya-focused job matching system designed to empower candidates rather than disqualify them. The platform uses a unique "Extract Once, Compute Many" architecture where LLM processing fires only at data ingestion time (CV upload and JD posting), while all subsequent matching happens deterministically via database queries at zero marginal cost. This design achieves an estimated monthly operating cost of approximately \$11 USD for processing 10,000 CVs against 1,000 active job postings.

The platform is specifically designed for the Kenyan employment landscape, where job seekers frequently hold multiple roles across different career functions simultaneously. Rather than penalizing candidates for diverse experience, the platform clusters related work experiences into career trajectories, allows candidates to select up to three focus areas, and ranks all matching jobs by relevance without ever hard-disqualifying a candidate. This philosophy of candidate empowerment is embedded in every layer of the system, from the normalization pipeline to the matching algorithm to the user interface.

The build plan is organized into five sequential phases spanning approximately 10 weeks. Each phase is self-contained with clear deliverables, acceptance criteria, and dependencies on previous phases. The first phase establishes the technical foundation including project scaffolding, database schema, and authentication. Subsequent phases layer on the data pipeline, matching engine, candidate-facing interface, and finally the admin portal, privacy compliance features, and production readiness. This phased approach allows for continuous testing and validation at each stage, reducing the risk of late-stage integration failures.

Metric	Value
Monthly LLM Cost	~\$11 USD (10K CVs + 1K jobs)
Matching Cost	\$0 per match (database queries only)
Total Timeline	5 phases, approximately 10 weeks
Tech Stack	Next.js 15, TypeScript, MySQL 8.0, Prisma, NextAuth.js
Compliance	Kenya Data Protection Act (2019)
LLM Provider	Gemini 1.5 Flash (~\$0.001 per extraction)

Table 1: Platform Key Metrics

2. Architecture Overview

The platform follows a monolithic Next.js architecture with a clear separation between the frontend presentation layer and the backend API routes. All matching logic runs on the server side, and a cron-based background job handles the periodic computation of match scores between candidates and jobs. This architecture was chosen for its simplicity, deployability, and cost efficiency, avoiding the complexity of microservices while still supporting the scale requirements of the Kenyan job market.

2.1 System Components

The system comprises seven major components that work together in a pipeline. Understanding these components and their interactions is essential before beginning any development phase.

Component	Technology	Description
Text Extraction	pdf-parse / mammoth	Extracts raw text from uploaded CV and JD files
LLM Integration	Gemini 1.5 Flash API	Parses raw text into structured JSON (fires once per document)
Normalization	Word-boundary regex + enums	Maps variations to canonical values, de-duplicates skills
Matching Engine	Function filter + score calc	Pre-computes matches using DB-indexed function queries
Background Jobs	GitHub Actions / Vercel Cron	Runs matching every 5 minutes on new/updated data
Authentication	NextAuth.js (Google + Email)	Session management, role-based access control
Privacy Layer	Consent, export, soft-delete	DPA-compliant data handling with 30-day deletion grace period

Table 2: System Components Overview

2.2 Data Flow

The data flow through the system follows two parallel paths that converge at the matching engine. The candidate path begins with CV upload, proceeds through LLM extraction and normalization, and stores structured data in the MySQL database. The job path follows the same pattern but supports four distinct input methods: paste JD text (with LLM), upload file (with LLM), paste JSON (no LLM), or fill a form directly (no LLM). Both paths deposit their structured outputs into the database, where the background matching cron job picks them up and computes match scores.

A critical design decision is that LLM is only invoked when new data enters the system. Once a CV or JD has been extracted and normalized, all subsequent matching is performed through pure database queries and deterministic scoring functions. This means the per-match cost is effectively zero, and the candidate dashboard can display results instantly without any LLM latency. The cron job runs every five minutes and only processes jobs posted within the last hour, using function-indexed queries to avoid the N-squared scalability trap of comparing every candidate against every job.

2.3 Matching Philosophy

The matching algorithm is designed around five core principles: inclusiveness (never hiding jobs based on missing data), function-based filtering (primary match on job function), ranked results (best matches appear first), explainability (each match shows why it was recommended), and user control (only selected career trajectories are used for matching). The scoring system allocates 100 points across five dimensions: job title match (40 points), skills match (35 points), education level (15 points), education field relevance (5 points), and experience years (10 points). Importantly, missing data never disqualifies a candidate; it simply results in a lower score rather than exclusion.

3. Phase 1: Foundation and Infrastructure

PHASE 1

Foundation and Infrastructure

Week 1-2 (10 working days)

Goals

- Initialize the Next.js 15 project with TypeScript and configure the development environment
- Set up MySQL 8.0 database with the complete schema including all tables, indexes, and relationships
- Implement authentication and authorization using NextAuth.js with Google and email providers
- Build the privacy and consent infrastructure including consent recording and soft-delete support
- Establish the enum normalization system with word-boundary matching to prevent false positives
- Configure the project structure, environment variables, and deployment pipeline for Vercel

3.1 Project Initialization

The first task is to scaffold the Next.js 15 project using the App Router architecture. The project structure should follow the technical documentation layout with separate directories for API routes, library utilities, models, types, scripts, and Prisma schema files. Key dependencies to install include Next.js 15, React 18, Prisma 5, NextAuth.js 4.24, Google Generative AI SDK, pdf-parse, mammoth, Zod for validation, and bcryptjs for password hashing. TypeScript should be configured with strict mode enabled from the start to catch type errors early.

Environment variables must be configured for all external services: the Gemini API key, database connection string (DATABASE_URL), NextAuth secret, and cron job authentication token. A .env.local file should be created with placeholder values, and a .env.example file should be committed to version control for documentation purposes. The development script should include database migration commands, seed data generation, and a local development server startup sequence.

3.2 Database Schema

The MySQL database requires nine core tables. The candidates table stores profile information, authentication credentials (password hash, auth provider, auth ID), and account status flags (is_active, deleted_at for soft-delete support). The candidate_education table is a child table supporting multiple qualifications per candidate, which is critical for the Kenyan market where candidates frequently hold both a degree and professional certifications like CPA or ACCA.

The candidate_clusters table stores the LLM-extracted work experience clusters, each with a normalized function enum, job titles array, skills array, years of experience, and a selected boolean flag that the candidate uses to choose their focus trajectories. The jobs table mirrors the candidate structure with normalized function, sector,

and education level enums. The `job_matches` table serves as the pre-computed cache of matching results with a unique constraint on `(candidate_id, job_id)` and a descending index on `match_score` for fast retrieval.

Three additional tables support privacy compliance: `consent_records` tracks every consent action with the consent type, version, IP address, and user agent; `deletion_requests` manages account deletion workflows with a status enum (pending, processing, completed, cancelled) and a `deleted_data` JSON field for audit trails; and `export_requests` tracks data export requests with an expiry date for the generated download link. The `parse_failures` table logs any LLM extraction errors with the raw text and error message for manual review and resolution.

Table	Purpose	Key
<code>candidates</code>	Profile, auth, account status	PRIMARY
<code>candidate_education</code>	Multiple qualifications per candidate	FK -> candidates
<code>candidate_clusters</code>	Work experience clusters (matching)	FK -> candidates
<code>candidate_extras</code>	Certifications, referees, languages	FK -> candidates
<code>jobs</code>	Job postings with normalized enums	PRIMARY
<code>job_matches</code>	Pre-computed match results	FK -> candidates, jobs
<code>consent_records</code>	DPA consent audit trail	FK -> candidates
<code>deletion_requests</code>	Account deletion workflow	FK -> candidates
<code>export_requests</code>	Data export workflow	FK -> candidates
<code>parse_failures</code>	LLM error logging	PRIMARY

Table 3: Database Schema Tables

3.3 Authentication

`NextAuth.js` provides the authentication layer with support for both Google OAuth and email/password credentials. The implementation requires a route handler at `app/api/auth/[...nextauth]/route.ts` that configures the providers and an authorize callback that validates credentials against the database. A `middleware.ts` file should enforce authentication on all protected routes, checking for the session token and redirecting unauthenticated users to the login page. Role-based access control should be implemented using the session token, distinguishing between "admin" and "candidate" roles.

3.4 Normalization System

The normalization system is critical for data quality and must be implemented before any data flows into the system. It consists of three enum mapping dictionaries (`EDUCATION_LEVEL_ENUMS` with 6 canonical values, `FUNCTION_ENUMS` with 16 categories, and `SECTOR_ENUMS` with 21 industry sectors) and a `normalizeWithWordBoundaries` function that uses regex word boundaries to prevent false-positive matches. Without word-boundary matching, the string "hr" would incorrectly match "research" or "architecture" because the naive `includes()` check finds substrings. The implementation uses `RegExp` with `\b` anchors to ensure only whole-word matches are accepted.

3.5 Deliverables and Acceptance Criteria

ID	Deliverable	Acceptance
D1.1	Next.js project scaffolded with TypeScript, all dependencies installed	Dev server starts without errors
D1.2	MySQL database created with all 10 tables, indexes, and constraints	Prisma migrate succeeds, schema matches documentation
D1.3	NextAuth.js login/signup working with Google and email	Can create account, login, logout, session persists
D1.4	Consent recording on signup and CV upload	Consent records written with correct type and version
D1.5	Normalization pipeline with word-boundary matching	Unit tests pass for all enum edge cases
D1.6	Environment variables and deployment config	Deploy preview to Vercel succeeds

Table 4: Phase 1 Deliverables

4. Phase 2: Data Pipeline

PHASE 2

Data Pipeline (CV and JD Processing)

Week 3-4 (10 working days)

Goals

- Implement the LLM integration with Gemini 1.5 Flash for CV and JD extraction
- Build the complete CV upload flow with text extraction, LLM parsing, normalization, and storage
- Build the job posting flow supporting all four input methods (paste, upload, JSON, form)
- Implement the field similarity mapping system with the RELATED_FIELDS matrix
- Add LLM error handling with retry logic, exponential backoff, and manual review logging
- Implement the trajectory selection API with re-matching on preference change

4.1 LLM Integration

The LLM integration layer wraps the Google Generative AI SDK with a `parseWithRetry` function that attempts up to three retries with exponential backoff (1s, 2s, 4s delays). On each attempt, the function sends the extraction prompt with the raw text, parses the JSON response, and performs basic validation checking for required fields. If all retries fail, the function logs the failure to the `parse_failures` table for manual review and returns an error object with `_error`, `_message`, and `_raw_text` fields. This ensures that no extraction failure is silently lost, and the admin can review and manually fix problematic documents.

4.2 CV Upload Flow

The CV upload endpoint accepts multipart form data with the CV file and a consent confirmation. The flow proceeds through five stages: text extraction (pdf-parse for PDF files, mammoth for DOCX), LLM parsing with retry, normalization of education levels and work experience functions, database storage of the candidate profile and all child records (education array, work experience clusters, extras), and consent recording. The education array is stored in the `candidate_education` table as separate rows, each linked to the candidate via foreign key. Work experience clusters are similarly stored as individual rows in the `candidate_clusters` table with all skills and job titles serialized as JSON arrays. After successful upload, the system triggers an initial matching computation for the new candidate against all active jobs.

4.3 Job Posting (Four Input Methods)

The job posting endpoint uses an `input_type` discriminator to route processing through the appropriate path. For "paste" type, the raw JD text is sent to the LLM for extraction. For "upload" type, the file is first processed through text extraction then through LLM extraction. For "json" type, the pre-structured JSON is validated against

the job schema using Zod and then normalized without any LLM call. For "form" type, the already-structured form data is normalized directly. This conditional LLM usage is a key cost optimization: power users who paste JSON or fill forms avoid LLM costs entirely, while casual users who paste raw text or upload files get automatic extraction.

4.4 Field Similarity Mapping

The field similarity mapping replaces the naive `String.includes()` approach with a three-tier matching system. Tier 1 checks for exact string match. Tier 2 looks up both fields in the `RELATED_FIELDS` matrix, which contains 12 category groups covering Business, Computing, Engineering, Health, Education, Economics, Law, Arts, Sciences, Hospitality, Construction, and Media. If both fields belong to the same category group, they are considered related. Tier 3 performs partial-word matching on multi-word field names, checking for common significant words (length greater than 3 characters) while filtering out common stopwords like "and", "of", "the", "science", and "studies". This ensures that "Commerce" matches "Business Administration" (both in the Business group) but does not match "Business Journalism" (Journalism is in the Media group).

4.5 Deliverables and Acceptance Criteria

ID	Deliverable	Acceptance
D2.1	LLM integration with retry logic and error logging	3 retries with backoff, failures logged to DB
D2.2	CV upload: extract, parse, normalize, store, consent	Upload PDF/DOCX, parsed data stored correctly
D2.3	Job posting: all 4 input methods working	Paste, upload, JSON, form all produce valid job records
D2.4	Field similarity mapping with 12-category matrix	All 12 test cases pass (Commerce-Business Admin true, etc.)
D2.5	Trajectory selection API with max-3 validation	PUT endpoint validates limit, triggers re-match
D2.6	Consent flow integrated into CV upload	Upload rejected without consent checkbox

Table 5: Phase 2 Deliverables

5. Phase 3: Matching Engine

PHASE 3

Matching Engine

Week 5-6 (10 working days)

Goals

- Implement the complete scoring algorithm with all five dimensions (title, skills, education, field, experience)
- Build the scalable background matching cron job using function-indexed queries
- Implement explanation generation for every match to provide transparency to candidates
- Build the re-matching system that triggers when candidates change trajectory preferences
- Implement the match results API endpoint for the candidate dashboard to consume
- Test the matching engine with realistic Kenyan CV and JD data for accuracy validation

5.1 Scoring Algorithm

The matching algorithm operates in two stages. The first stage is a function-based gateway filter: a job is only shown to a candidate if at least one of the candidate's selected career clusters has a matching job function enum. This filter is implemented as a database JOIN between the candidate_clusters table (filtered by selected=TRUE and the specific function enum) and the jobs table. Jobs that pass the gateway filter proceed to the second stage, which calculates a weighted score across five dimensions.

The job title dimension (40 points) checks for exact match first, awarding full points if any of the cluster's job titles exactly equals the job title. If no exact match exists, it falls back to keyword overlap scoring, counting shared words between the job title and the cluster's title array, capped at 20 points. The skills dimension (35 points) calculates the percentage of required skills present in the candidate's skill set, using case-insensitive exact matching. The education level dimension (15 points) compares the candidate's highest education level against the job requirement using a tiered scoring system: full points if the candidate exceeds the requirement, 70% if one level below, 30% if two levels below, and zero otherwise. The education field bonus (5 points) uses the field similarity mapping to check if any of the candidate's education fields are related to the job's required field. The experience dimension (10 points) calculates the ratio of candidate years to required years, capped at 1.0 to prevent over-experience penalties.

Dimension	Points	Method	Notes
Job Title Match	40	Exact or partial keyword overlap	Highest weight - role alignment
Skills Match	35	Percentage of required skills present	Percentage of required skills found

Dimension	Points	Method	Notes
Education Level	15	Candidate meets or exceeds requirement	Tiered: 100%, 70%, 30%, 0%
Education Field	5	Bonus for related field	Uses RELATED_FIELDS matrix
Experience Years	10	Candidate years vs. required years	Ratio capped at 1.0

Table 6: Matching Score Dimensions

5.2 Background Matching Cron Job

The background matching job is designed for scalability from the outset. Rather than fetching all candidates and comparing against all jobs (an $O(N*M)$ operation), the cron job uses a function-indexed query strategy. For each new or recently updated job, it queries only the candidates whose selected clusters match the job's function enum. This reduces the comparison set dramatically: instead of comparing 10,000 candidates against every job, the system only evaluates the subset of candidates who have indicated relevant experience in that specific function.

The cron job implementation supports two deployment options. The primary option uses GitHub Actions with a cron schedule of `*/* * * * *` (every 5 minutes) and a manual `workflow_dispatch` trigger for on-demand execution. The script uses `npx ts-node` to run the TypeScript matching script, which queries jobs posted in the last hour, performs the function-indexed candidate lookup, calculates scores, and upserts results into the `job_matches` table using `ON DUPLICATE KEY UPDATE`. The secondary option uses Vercel Cron Jobs with an API endpoint protected by a Bearer token authentication check. Both options require the `CRON_SECRET` environment variable for authorization.

5.3 Explanation Generation

Every match record includes a human-readable explanation string that tells the candidate why the job was recommended. The explanation starts with the primary career identity (e.g., "This job matches your experience as an Admin Assistant"), then appends matched skills (up to 3 named skills plus a count of additional matches). This explanation is generated at match computation time and stored in the explanation column of the `job_matches` table, so it requires no computation when the candidate views their dashboard. The explanation provides transparency and builds trust in the matching system.

5.4 Deliverables and Acceptance Criteria

ID	Deliverable	Acceptance
D3.1	Complete scoring algorithm (all 5 dimensions)	Scores fall within 0-100 range, edge cases handled
D3.2	Background cron job with function-indexed queries	GitHub Actions cron runs every 5 min, Vercel fallback works
D3.3	Match explanation generation	Every match record has human-readable explanation
D3.4	Trajectory change triggers re-matching	Changing trajectories deletes old matches, creates new ones

ID	Deliverable	Acceptance
D3.5	Match results API endpoint	GET /candidates/[id]/matches returns ranked matches
D3.6	Accuracy validation with real data	10 sample Kenyan CVs produce expected match results

Table 7: Phase 3 Deliverables

6. Phase 4: User Interface (Candidate Experience)

PHASE 4

User Interface (Candidate Experience)

Week 7-8 (10 working days)

Goals

- Build the landing page with value proposition and sign-up flow
- Build the candidate onboarding flow: CV upload, trajectory selection, and consent
- Build the candidate dashboard with ranked matches, filters, and sorting
- Build the job detail page with match explanation, skill breakdown, and apply/save actions
- Implement location display as informational context (never as a filter)
- Ensure responsive design for mobile-first access given the Kenyan mobile usage patterns

6.1 Landing Page

The landing page communicates the platform's core value proposition: "We open doors, we do not close them." It should feature a clean, uncluttered design with a clear call-to-action for candidate registration and a separate entry point for admin login. The page should briefly explain how the platform works (upload CV, get matched, apply for jobs) and address common concerns about data privacy. Given that a significant portion of Kenyan job seekers access the internet primarily through mobile devices, the landing page must be fully responsive with a mobile-first design approach. Load time should be optimized since many users will be on slower mobile connections.

6.2 Candidate Onboarding

The onboarding flow guides new candidates through three steps after account creation. First, the CV upload step accepts PDF or DOCX files, displays a consent checkbox for data processing, and triggers the extraction pipeline. Second, the trajectory selection step presents the LLM-extracted work experience clusters with their functions, years of experience, representative companies, and key skills. Candidates can select up to three clusters to focus on, with a clear explanation that changing these preferences later is possible. Third, the system triggers initial matching and redirects to the dashboard. Each step should have a "Skip for Now" option so that candidates can complete onboarding quickly and return to configure preferences later.

6.3 Candidate Dashboard

The candidate dashboard is the primary engagement surface of the platform. It displays the candidate's selected focus areas at the top, followed by a ranked list of matching jobs. Each job card shows the match percentage, company name, sector, location (as informational context, never as a filter), and the match explanation snippet. The dashboard should support basic filtering (by sector, job type, match score range) and sorting (by score, date,

company). Location information is displayed as contextual data: for example, a candidate in Nairobi seeing a Mombasa job would see the distance or a note like "You are in Nairobi - 480km away." This follows the platform philosophy that location is the candidate's decision, not the platform's filter.

6.4 Job Detail Page

The job detail page provides a comprehensive view of a matched job alongside the match explanation. The page is divided into four sections: the job header (title, company, sector, location, type), the match analysis (score breakdown showing which dimensions matched and which did not), the full job description with requirements, and action buttons (Apply Now, Save Job, View Similar Jobs). The match analysis section should be visually prominent, using a checklist format with green checkmarks for matched criteria and warning indicators for missing skills. This transparency helps candidates understand their fit for the role and identify skill gaps to address before applying.

6.5 Deliverables and Acceptance Criteria

ID	Deliverable	Acceptance
D4.1	Landing page with sign-up flow	Mobile responsive, loads in under 3 seconds on 3G
D4.2	Onboarding: CV upload + trajectory selection	Can upload CV, see clusters, select 3, skip available
D4.3	Candidate dashboard with ranked matches	Matches display in order, filters work, location shows as info
D4.4	Job detail page with match explanation	Score breakdown, skill checklist, apply/save buttons work
D4.5	Responsive design across all pages	Works on mobile (375px), tablet, and desktop
D4.6	Navigation: profile, applications, privacy	All nav links point to functional pages

Table 8: Phase 4 Deliverables

7. Phase 5: Admin Portal, Privacy, and Production

PHASE 5

Admin Portal, Privacy, and Production

Week 9-10 (10 working days)

Goals

- Build the admin dashboard with the four-method job posting interface
- Implement the privacy center with data export and account deletion workflows
- Write unit tests and integration tests for the matching engine and API endpoints
- Perform performance optimization, query analysis, and caching implementation
- Deploy to production with monitoring, error tracking, and security hardening
- Complete API documentation and user-facing guides

7.1 Admin Dashboard

The admin dashboard serves two primary functions: job posting and application management. The job posting interface presents four clearly labeled input methods in a tabbed or card-based layout. The "Paste JD" tab provides a large text area where admins can paste raw job descriptions, with an "Auto-Extract" button that triggers LLM parsing and populates the form fields below. The "Upload File" tab accepts PDF and DOCX uploads with the same auto-extract flow. The "Paste JSON" tab targets power users who generate structured JSON from external tools like ChatGPT or DeepSeek, with JSON validation and error highlighting. The "Fill Form" tab provides the complete manual entry form with dropdown selects for all enum fields (sector, function, job type, education level), text inputs for required skills (with tag-style input), and date pickers for deadlines.

After a job is posted through any method, the system normalizes the data, stores it in the database, and triggers the background matching job. The admin should see a confirmation message with the job ID and a link to preview the job listing. The application management section shows all submitted applications with the candidate's profile, match score, and application status (pending, reviewed, shortlisted, rejected, hired). Admins can update application statuses and add internal notes visible only to the admin team.

7.2 Privacy Center

The privacy center provides three DPA-mandated features. The data export endpoint compiles all of a candidate's stored data (profile, education, work experience clusters, extras) into a single JSON object, creates an export record with a 7-day expiry, and returns the data for download. The account deletion endpoint requires the user to type "DELETE" for confirmation, initiates a soft-delete by setting `is_active` to `FALSE` and `deleted_at` to the current timestamp, removes the candidate from all match results, and creates a deletion request record with a 30-day pending status. After 30 days, a monthly cleanup cron job permanently purges soft-deleted accounts and

all their associated data. The consent management section shows a history of all consent actions (signup, CV upload, marketing, data processing) with timestamps and versions.

7.3 Testing Strategy

Testing should cover three levels. Unit tests (using Jest) should validate individual functions: the normalization pipeline (ensuring "BSc" maps to "Bachelor", "hr" does not match "research"), the scoring algorithm (ensuring scores are within 0-100, edge cases for missing data produce partial scores rather than zero), and the field similarity mapping (the 12 test cases from the technical documentation). Integration tests should validate API endpoints end-to-end: upload a CV, verify database records, post a job, verify matching results. End-to-end tests should simulate a complete user journey: register, upload CV, select trajectories, view matches, apply for a job, export data, delete account.

7.4 Production Readiness

Production deployment requires several final checks. Database query optimization should identify slow queries (particularly the matching cron job) and add appropriate composite indexes. Security hardening should include rate limiting on API endpoints, CSRF protection, input sanitization, and SQL injection prevention (mitigated by Prisma's parameterized queries). Error tracking should be configured using a service like Sentry or equivalent, with alerts for LLM extraction failures exceeding a threshold. The deployment should target Vercel for the frontend and API routes, with a managed MySQL instance (PlanetScale, Amazon RDS, or equivalent) for the database. Environment variables should be configured through Vercel's dashboard, never committed to the repository.

7.5 Deliverables and Acceptance Criteria

ID	Deliverable	Acceptance
D5.1	Admin dashboard with 4-method job posting	All 4 methods create valid job records
D5.2	Privacy center: export, delete, consent history	Export returns JSON, delete soft-removes account
D5.3	Unit tests: normalization, scoring, field mapping	All tests pass, coverage exceeds 80%
D5.4	Integration tests: CV upload, job post, matching	Full pipeline tested end-to-end
D5.5	Performance: matching cron under 30 seconds	10K candidates, 100 jobs matched in under 30s
D5.6	Production deployment with monitoring	Live on Vercel, error tracking active

Table 9: Phase 5 Deliverables

8. Cross-Phase Dependencies

Understanding the dependencies between phases is critical for planning sprints and allocating developer resources. The following dependency map shows which deliverables from earlier phases are required before specific tasks in later phases can begin. Tasks within the same phase can generally be parallelized, but cross-phase dependencies create a sequential ordering constraint.

Depends On	Enables	Reason
D1.2 (Database Schema)	D2.2 (CV Upload)	Cannot store CV data without tables
D1.3 (Authentication)	D2.2 (CV Upload)	CV upload requires authenticated session
D1.5 (Normalization)	D2.2 (CV Upload)	CV data must be normalized before storage
D2.1 (LLM Integration)	D2.3 (Job Posting)	Job parsing reuses the LLM retry logic
D2.4 (Field Mapping)	D3.1 (Scoring Algorithm)	Education field bonus uses field mapping
D2.2/D2.3 (Data Storage)	D3.2 (Cron Job)	Matching requires candidate and job data in DB
D1.2 (Database Schema)	D3.2 (Cron Job)	job_matches table must exist
D3.5 (Match API)	D4.3 (Dashboard)	Dashboard fetches matches from the API
D3.3 (Explanations)	D4.4 (Job Detail)	Job detail page shows explanation text
D1.4 (Consent Records)	D5.2 (Privacy Center)	Privacy features need consent infrastructure
D4.3 (Dashboard)	D5.3 (Testing)	Integration tests need a functional UI

Table 10: Cross-Phase Dependency Map

8.1 Parallelization Opportunities

While the phases are sequential, individual tasks within each phase can be parallelized across a team of two to three developers. In Phase 1, the database schema, authentication, and normalization system can be developed in parallel since they are independent components. In Phase 2, the CV upload flow and job posting flow share the LLM integration layer but diverge in their specific handling, so one developer can focus on the CV path while another works on the JD path. In Phase 3, the scoring algorithm and the cron job infrastructure can be developed in parallel if the database schema is already in place. In Phase 4, the landing page, onboarding flow, dashboard, and job detail page can be split across developers since they are separate UI components that consume the same API endpoints.

9. Risk Register and Mitigations

Every project carries risks that could delay delivery or reduce quality. This section identifies the most significant risks for the build plan and describes specific mitigation strategies that should be proactively implemented rather than reactively addressed.

ID	Risk	Likelihood	Impact	Description	Mitigation
R1	LLM Extraction Inaccuracy	HIGH	MEDIUM	The LLM may misclassify functions or extract incorrect data from poorly formatted Kenyan CVs	The word-boundary normalization catches common misclassifications. The <code>parse_failures</code> table logs all failures for manual review. The consent flow allows candidates to edit their extracted profile. Budget 2-3 days for manual review of the first 100 extractions.
R2	Normalization False Positives	MEDIUM	LOW	The normalization enum matching may incorrectly map variants	Word-boundary regex prevents substring false matches. Unit tests cover all documented edge cases. The <code>RELATED_FIELDS</code> matrix with stopwords filtering handles field similarity.
R3	Matching Scalability	MEDIUM	MEDIUM	The N-squared matching problem as candidates and jobs grow	Function-indexed queries reduce the comparison set dramatically. Incremental matching (only jobs from the last hour) limits the per-run workload. Composite indexes on function and selected fields ensure query performance. Monitor query times and add query caching if needed.
R4	DPA Compliance Gaps	LOW	HIGH	The platform handles personal data subject to Kenya's Data Protection Act	Consent flows are implemented at signup and CV upload. Data export and deletion APIs are available. Soft-delete with 30-day grace period is implemented. Monthly cleanup cron job handles retention. Consult a DPA expert before launch for compliance audit.
R5	Kenyan Mobile Network Latency	MEDIUM	LOW	Users on slow mobile connections may have poor experience	Mobile-first responsive design minimizes bandwidth usage. API responses should be paginated and compressed. Static assets should be served through Vercel's CDN. Landing page should load in under 3 seconds on 3G connections.

Table 11: Risk Register

10. Team Composition and Effort Estimates

10.1 Recommended Team

The recommended team composition for this build is two full-stack developers with experience in Next.js, TypeScript, and MySQL. Both developers should be comfortable working across the full stack since the project uses Next.js API routes rather than a separate backend service. One developer should have stronger frontend/UI skills (for Phases 4 and 5), while the other should have stronger backend/data skills (for Phases 2 and 3). A part-time DevOps resource should be available for deployment and infrastructure setup in Phase 5. An optional QA resource can assist with test writing and manual testing in Phases 3-5.

Role	Phases	Focus Area	Commitment
Full-Stack Developer 1	Phases 1-5	Backend focus: DB, API, matching, cron jobs	Full-time, 10 weeks
Full-Stack Developer 2	Phases 1-5	Frontend focus: UI components, responsive design	Full-time, 10 weeks
DevOps (part-time)	Phase 5 only	Vercel deployment, MySQL provisioning, monitoring	Part-time, 2 weeks
QA (optional)	Phases 3-5	Test writing, manual testing, regression testing	Part-time, 4 weeks

Table 12: Recommended Team Composition

10.2 Effort Summary

Phase	Timeline	Resources	Effort
Phase 1: Foundation	Week 1-2	2 devs	10 person-days
Phase 2: Data Pipeline	Week 3-4	2 devs	20 person-days
Phase 3: Matching Engine	Week 5-6	2 devs	20 person-days
Phase 4: Candidate UI	Week 7-8	2 devs	20 person-days
Phase 5: Admin + Production	Week 9-10	2 devs + part-time ops	22 person-days
		TOTAL	92 person-days

Table 13: Effort Summary

10.3 Milestone Checkpoints

Each phase concludes with a formal milestone review where the deliverables are verified against the acceptance criteria defined in each phase section. Milestone reviews should include a demo of the working features, a review of any test failures or known issues, and a go/no-go decision for proceeding to the next phase. The following milestones are defined:

Milest one	Checkpoint	Exit Criteria
M1	End of Phase 1	Project runs locally, auth works, DB schema is complete, normalization tests pass
M2	End of Phase 2	CV upload produces correct parsed data, job posting works with all 4 methods, consent flows work
M3	End of Phase 3	Matching engine produces correct scores, cron job runs every 5 min, re-matching works
M4	End of Phase 4	Candidate dashboard shows ranked matches, responsive on mobile, job detail page works
M5	End of Phase 5	Admin can post jobs, privacy center works, tests pass, deployed to production

Table 14: Milestone Checkpoints